

DAFTAR PERTANYAAN KEMUNGKINAN DOSEN PENGUJI

Sidang Skripsi: Pengembangan High Performance Application MBLB di Kabupaten Lumajang

Azka Kasmito Putra — NIM 2241720216

● KATEGORI 1: METODOLOGI & VALIDITAS PENELITIAN

Q1: Mengapa memilih desain quasi-experimental, bukan eksperimen murni?

Penelitian ini menggunakan quasi-experimental karena tidak memungkinkan untuk melakukan random assignment pada subjek penelitian. Sistem existing E-PASIR adalah sistem produksi yang sudah berjalan — saya tidak dapat memodifikasinya secara acak. Desain quasi-experimental (Pretest-Posttest Non-Equivalent Control Group) dipilih karena memungkinkan perbandingan yang valid antara dua kondisi yang tidak dapat dirandomisasi, dengan cara mengontrol kondisi pengujian secara ketat: lingkungan identik (localhost, instance PostgreSQL yang sama), skenario JMeter identik, dan tiga kali repetisi untuk mengurangi variansi.

Q2: Mengapa pengujian dilakukan di localhost, bukan di server produksi?

Tujuan utama adalah memastikan validitas perbandingan antara sistem existing dan sistem baru. Jika pengujian dilakukan di server produksi, ada variabel tidak terkontrol yang dapat mempengaruhi hasil: latensi jaringan, beban server lain, dan konfigurasi infrastruktur yang berbeda. Dengan pengujian di localhost pada kondisi identik, saya dapat mengisolasi variabel yang benar-benar relevan: perbedaan arsitektur dan implementasi kedua sistem. Ini adalah keputusan metodologis yang disengaja, bukan keterbatasan teknis. Saran untuk penelitian lanjutan adalah melakukan pengujian di lingkungan staging yang merepresentasikan produksi.

Q3: Mengapa menggunakan paired t-test? Apa asumsinya?

Paired t-test dipilih karena data bersifat berpasangan: setiap endpoint yang sama diuji pada kedua sistem (7 pasangan data). Asumsi paired t-test adalah: (1) data bersifat continuous — terpenuhi, karena response time dan throughput adalah data kontinu; (2) populasi berdistribusi normal atau ukuran sampel cukup besar — dengan 9.300 sampel per sistem, Central Limit Theorem menjamin normalitas distribusi rata-rata; (3) observasi berpasangan — terpenuhi karena setiap endpoint diuji pada kedua sistem.

Selain t-test, saya juga menghitung Cohen's d untuk effect size, karena signifikansi statistik saja tidak cukup — $d > 0,8$ menunjukkan perbedaan bermakna secara praktis.

Q4: Apa itu Cohen's d dan mengapa penting?

Cohen's d adalah ukuran besar efek (effect size) yang mengukur seberapa besar perbedaan antara dua kelompok dalam satuan standar deviasi. Interpretasinya: $d < 0,2$ (efek diabaikan), 0,2–0,5 (kecil), 0,5–0,8 (sedang), $> 0,8$ (besar/large). Pada penelitian ini, Cohen's d untuk response time = 1,964 dan throughput = 2,371 — keduanya jauh di atas 0,8. Ini penting karena p-value saja bisa menyesatkan: dengan sampel yang sangat besar, perbedaan kecil pun bisa signifikan secara statistik. Cohen's d memastikan bahwa perbedaan yang terukur memang bermakna secara praktis, bukan hanya artefak ukuran sampel.

● KATEGORI 2: TEKNIS SISTEM & OPTIMASI PERFORMA

Q5: Bagaimana Virtual Threads Java 21 bekerja dan mengapa lebih efisien dari platform threads biasa?

Platform threads (thread OS) bersifat heavyweight — setiap thread membutuhkan ~1 MB stack memory. Ketika jumlah concurrent users tinggi, platform threads akan kehabisan kapasitas dan permintaan antri di queue, menyebabkan response time meningkat drastis (efek queueing). Virtual Threads adalah thread ringan yang dikelola oleh JVM, bukan OS. Satu platform thread dapat menangani ribuan virtual threads. Ketika virtual thread menunggu I/O (misalnya query database), JVM akan "memarkir" virtual thread tersebut dan menggunakan platform thread yang sama untuk menjalankan virtual thread lain — ini disebut continuation/preemption. Hasilnya: konkurensi tinggi tanpa overhead memori atau antrean thread yang panjang. Diaktifkan dengan satu baris konfigurasi: `spring.threads.virtual.enabled=true`.

Q6: Apa itu HikariCP dan mengapa konfigurasi maximum-pool-size = 50?

HikariCP adalah connection pool library untuk Java yang dikenal sebagai yang tercepat dan paling efisien. Tanpa connection pool, setiap HTTP request harus membuat koneksi baru ke database (overhead 20–100 ms per koneksi). Dengan HikariCP, koneksi sudah tersedia dalam pool dan siap digunakan. Maximum-pool-size 50 artinya sistem dapat mempertahankan maksimal 50 koneksi simultan ke PostgreSQL. Nilai ini dipilih berdasarkan spesifikasi lingkungan lokal dan rekomendasi HikariCP: terlalu kecil menyebabkan antrean, terlalu besar menyebabkan overhead di sisi database. Untuk produksi, nilai optimal perlu diturunkan/dinaikan berdasarkan profil beban nyata.

Q7: Bagaimana Caffeine Cache bekerja dan apa risiko data stale?

Caffeine adalah in-memory cache library untuk Java, lebih efisien dari Guava Cache karena menggunakan algoritma W-TinyLFU untuk eviction. Cara kerja: ketika endpoint yang sama dipanggil, sistem mengecek cache terlebih dahulu (cache hit) — jika ada, data dikembalikan langsung dari memori tanpa query database; jika tidak ada (cache miss), sistem query database dan menyimpan hasilnya di cache. TTL (Time-To-Live) 30 detik artinya data di-cache selama 30 detik kemudian dihapus otomatis. Risiko data stale: selama 30 detik, data bisa berbeda dengan yang ada di database jika terjadi update. Untuk data pajak yang relatif statis dalam jangka pendek, 30 detik dapat diterima. Untuk endpoint yang sangat kritis terhadap konsistensi data real-time, cache harus di-invalidate secara manual setelah setiap operasi tulis (write-through atau cache invalidation on update).

Q8: Mengapa endpoint Login memiliki response time lebih tinggi (460 ms) dibanding endpoint bisnis (9–17 ms)?

Ini adalah desain yang disengaja untuk alasan keamanan. Login menggunakan BCrypt dengan cost factor 12 untuk hashing dan verifikasi password. BCrypt secara by-design dirancang lambat: cost factor 12 berarti $2^{12} = 4.096$ iterasi hashing. Ini mencegah serangan brute-force — bahkan jika penyerang mendapatkan hash password dari database, mereka perlu waktu sangat lama untuk memecahkannya. Trade-off: response time Login lebih lambat, tetapi operasi Login hanya terjadi sekali per sesi; setelahnya JWT access token digunakan untuk semua request. 460 ms masih jauh di bawah ambang 3.000 ms dan di bawah P99 694 ms — pengguna tidak akan merasakan perbedaan yang signifikan.

Q9: Apa itu ETag dan HTTP 304, dan bagaimana kontribusinya terhadap performa?

ETag (Entity Tag) adalah mekanisme HTTP caching berbasis fingerprint konten. Cara kerja: saat server pertama kali merespons request, ia menyertakan header `ETag: "hash-konten"`. Pada request berikutnya, klien mengirimkan `If-None-Match: "hash-konten"`. Jika konten tidak berubah, server merespons dengan HTTP 304 Not Modified (tanpa body) — menghemat bandwidth dan waktu transfer. Kontribusi ke performa: mengurangi overhead transfer data pada permintaan berulang untuk data yang sama. Ini relevan untuk endpoint yang datanya jarang berubah, seperti daftar pengajuan yang sama di-refresh berkali-kali oleh operator.

● KATEGORI 3: KEAMANAN SISTEM

Q10: Mengapa menggunakan dua token (access token 15 menit + refresh token 7 hari)?

Single long-lived token berisiko: jika token dicuri, penyerang memiliki akses selama umur token tersebut. Mekanisme dual token memisahkan dua concern: access token (15 menit) pendek umurnya untuk meminimalkan window eksploitasi jika dicuri; refresh token (7 hari) digunakan untuk mendapatkan access token baru tanpa login ulang, disimpan di database untuk memungkinkan revokasi. Jika perangkat pengguna hilang atau akun dikompromikan, administrator dapat merevoke refresh token di database — memaksa re-authentication pada semua perangkat. Access token yang sudah terbit tidak dapat di-revoke (stateless), tetapi umurnya singkat sehingga dampaknya terbatas.

Q11: Bagaimana mekanisme RBAC granular diimplementasikan?

RBAC (Role-Based Access Control) diimplementasikan menggunakan Spring Security dengan anotasi `@PreAuthorize` pada setiap method controller. Struktur: setiap Role (Admin BPRD, Operator, Wajib Pajak) memiliki kumpulan Permission tertentu (seperti `topup:create`, `topup:approve`, `user:view`). Ketika endpoint diakses, Spring Security memeriksa apakah JWT pengguna memiliki permission yang dibutuhkan sebelum method dieksekusi. Jika tidak, sistem langsung mengembalikan HTTP 403 Forbidden. Granularitas: berbeda dari RBAC kasar yang hanya memeriksa role, RBAC granular ini memeriksa permission spesifik — memungkinkan kontrol akses yang lebih halus dan fleksibel.

Q12: Bagaimana Audit Trail diimplementasikan secara asinkron dan mengapa asinkron?

Audit Trail diimplementasikan menggunakan Spring `@Async`: setiap operasi sensitif (login, pembuatan pengajuan, approval, dll.) memicu method `@Async` yang menyimpan log ke tabel `audit_logs` di database. Asinkron artinya proses pencatatan audit dilakukan di thread terpisah tanpa memblokir response ke pengguna — response HTTP langsung dikembalikan, sementara logging berjalan di background. Alasan asinkron: jika logging dilakukan sinkron (dalam thread yang sama), kegagalan insert audit log dapat menyebabkan rollback operasi bisnis utama, atau menambah latensi response. Dengan `@Async`, operasi bisnis tidak terganggu oleh logging.

● KATEGORI 4: HASIL & ANALISIS

Q13: Mengapa error rate sistem existing sangat tinggi (40,04%) dan mayoritas error adalah HTTP 401?

Dari 3.724 total error, 87,81% (3.270) adalah HTTP 401 Unauthorized, bukan error logika bisnis. Ini mengindikasikan bahwa manajemen sesi pada sistem existing tidak mampu mempertahankan status autentikasi di bawah beban konkuren tinggi. Kemungkinan penyebab: (1) token/session pada sistem existing memiliki masa hidup yang lebih pendek dari durasi pengujian, menyebabkan request ditolak karena token kedaluwarsa saat masih dalam antrean; (2) mekanisme session management mungkin menggunakan in-memory state yang tidak thread-safe di bawah konkurensi tinggi. Sisanya: 12,03% SocketTimeoutException (request tidak dapat dijawab dalam waktu timeout) dan 0,16% HTTP 500 (error internal server). Sistem baru tidak menghasilkan satu pun error dari ketiga kategori ini karena JWT stateless + Virtual Threads mengeliminasi seluruh akar masalah tersebut.

Q14: Bagaimana Anda memastikan pengujian JMeter fair dan tidak memihak sistem baru?

Beberapa langkah untuk memastikan fairness: (1) Skenario identik — jumlah concurrent users, ramp-up time, loop count, dan endpoint yang diuji persis sama untuk kedua sistem; (2) Lingkungan identik — localhost, instance database yang sama, tidak ada proses background berbeda; (3) Tiga kali repetisi dan dirata-ratakan — mengurangi variansi pengukuran akibat kondisi sesaat; (4) Threshold waktu identik — Duration Assertion 3.000 ms diterapkan sama; (5) Token disiapkan sebelum pengujian — baik sistem existing maupun baru disiapkan token autentikasinya sebelum skenario dimulai agar tidak ada overhead login tambahan; (6) Data di database sama — menggunakan dataset yang representatif dan identik untuk kedua sistem.

Q15: Apakah throughput 29,40 req/s realistic untuk kebutuhan BPPRD Lumajang?

Sangat realistis — bahkan jauh melebihi kebutuhan. BPPRD Lumajang mengelola 42 wajib pajak dengan rata-rata 41 transaksi per bulan. Jika semua transaksi terjadi dalam 8 jam kerja: $41 \text{ transaksi/bulan} \times 12 \text{ bulan} = 492 \text{ transaksi/tahun}$, atau sekitar 2 transaksi per hari. Target 1.000 transaksi per hari pun sudah sangat konservatif — ditetapkan untuk mengakomodasi skenario pertumbuhan ekstrem. $\text{Throughput } 29,40 \text{ req/s} \times 28.800 \text{ detik (8 jam)} = \sim 847.000 \text{ transaksi per hari}$. Ini berarti sistem memiliki kapasitas $423.000\times$ di atas kebutuhan aktual. Margin ini memastikan sistem mampu menangani lonjakan beban apapun di masa depan, termasuk ekspansi ke lebih banyak wajib pajak.

Q16: Apa yang menyebabkan error rate sistem existing sebesar 40% — apakah ini karena bug pada sistem existing?

Tidak sepenuhnya. Error rate 40% terutama disebabkan oleh ketidakmampuan arsitektur sistem existing menangani beban konkuren yang tinggi, bukan bug fungsional. Pada penggunaan normal (1-2 pengguna bersamaan), sistem existing mungkin berfungsi dengan baik. Masalah muncul saat beban disimulasikan dengan 300-1.500 concurrent users: (1) thread pool exhausted — sistem kehabisan kapasitas thread untuk memproses request baru; (2) koneksi database tidak cukup tanpa connection pool; (3) sesi autentikasi tidak dapat dipertahankan di bawah beban ini. Ini adalah masalah skalabilitas arsitektur (architectural scalability issue), bukan bug logika bisnis.

● KATEGORI 5: RUANG LINGKUP & BATASAN

Q17: Mengapa hanya fokus pada fitur Pengajuan Top Up, tidak seluruh fitur E-PASIR?

Ada dua alasan utama: (1) Pengajuan Top Up adalah fitur paling kritis secara operasional — ini adalah prasyarat agar wajib pajak dapat melakukan transaksi SKAB. Kegagalan fitur ini berarti aktivitas penambangan terhenti. Oleh karena itu, performa fitur ini paling berdampak langsung; (2) Keterbatasan waktu dan sumber daya penelitian skripsi — memilih satu ruang lingkup yang jelas dan terukur lebih baik daripada mencakup semua fitur secara dangkal. Infrastruktur keamanan dan performa yang dikembangkan (JWT, RBAC, Virtual Threads, HikariCP, Caffeine Cache) bersifat modular dan dapat diterapkan ke seluruh modul E-PASIR dalam pengembangan lanjutan.

Q18: Mengapa memilih stack Angular + Java Spring Boot, bukan melanjutkan dengan Nuxt.js + Golang?

Beberapa pertimbangan: (1) Java 21 Virtual Threads adalah fitur baru yang menjanjikan peningkatan performa konkuren secara signifikan — ini adalah kesempatan untuk mengintegrasikan teknologi terkini; (2) Spring Boot 3.2.5 memiliki ekosistem mature untuk optimasi performa (HikariCP built-in, Spring Cache abstraction, Spring Security untuk RBAC); (3) Angular dipilih karena type-safe (TypeScript) dan cocok untuk aplikasi enterprise dengan banyak form dan state management; (4) PostgreSQL dipertahankan untuk memastikan kompatibilitas data dan meminimalkan risiko migrasi. Perlu dicatat bahwa perbandingan ini bukan tentang membuktikan Go lebih buruk dari Java secara umum, melainkan tentang implementasi spesifik E-PASIR yang tidak menggunakan fitur optimasi yang tersedia.

● PERTANYAAN BONUS (TINGKAT LANJUT)

Q19: Bagaimana Cohen's $d = 1,964$ dihitung untuk response time?

Cohen's $d = (M1 - M2) / SD_{\text{pooled}}$, di mana $M1$ = mean sistem existing, $M2$ = mean sistem baru, dan SD_{pooled} = standar deviasi gabungan. Dengan data 7 pasangan endpoint: mean existing ≈ 7.667 ms, mean baru $\approx 26,61$ ms, selisih ≈ 7.640 ms. Standar deviasi pooled dihitung dari variansi kedua kelompok. Nilai $d = 1,964$ berarti rata-rata sistem baru berada 1,964 standar deviasi di bawah sistem existing — jauh di atas threshold large effect (0,8). Ini membuktikan bahwa perbedaan yang teramati bukan akibat variasi acak pengukuran.

Q20: Apakah ada risiko race condition pada Virtual Threads dan bagaimana mengatasinya?

Ya, Virtual Threads membawa potensi race condition yang sama seperti multi-threading pada umumnya. Mitigasi pada sistem ini: (1) operasi database menggunakan Spring Data JPA dengan transaction management (@Transactional) yang memastikan operasi database bersifat atomic; (2) operasi bisnis kritis seperti pembuatan pengajuan dan update saldo menggunakan pessimistic locking atau optimistic locking di level database; (3) Caffeine Cache dikonfigurasi thread-safe by default; (4) @Async untuk Audit Trail menggunakan dedicated thread pool terpisah dari thread pool utama, menghindari resource contention. Selain itu, pengujian JMeter dengan 100-1.500 concurrent users adalah stress test yang secara implisit menguji ketahanan terhadap race condition.

Dokumen ini disiapkan berdasarkan isi skripsi Azka Kasmito Putra (2026). Pelajari angka-angka spesifik di halaman 102-107 skripsi untuk menjawab pertanyaan detail.